

ECONOMICON

FinOps sin fricción, para quien no tiene tiempo de ser experto en FinOps.

Memoria del proyecto

Asistente FinOps para Azure — Propuesta

Alejandro Aguado · Paris Arcos · Lucía Mateo · Víctor Méndez

Trabajo Fin de Máster · Entrega prevista: octubre de 2026

Índice

A. Introducción y resumen ejecutivo **2**

A.1. Resumen ejecutivo . . . 3

B. Caso de negocio **3**

B.1. La oportunidad . . . 3

B.2. Audiencia objetivo . . . 3

B.3. Análisis de mercado . . . 3

B.4. Justificación de la solución . . . 4

B.4.1. Diferenciador central . . . 4

B.5. Viabilidad . . . 4

B.5.1. Técnica . . . 4

B.5.2. Operativa . . . 4

B.5.3. Económica . . . 5

B.6. Impacto esperado . . . 5

B.7. Costes y supuestos . . . 5

B.7.1. Coste de la inteligencia artificial . . . 5

B.7.2. Coste de infraestructura . . . 6

B.8. Riesgos . . . 6

B.8.1. Riesgos técnicos . . . 6

B.8.2. Riesgos económicos . . . 6

B.8.3. Riesgos de contexto . . . 6

C. Contribución individual **7**

C.1. Roles rotatorios . . . 7

C.2. Quién entregó qué . . . 7

C.3. Trazabilidad y límites de la evidencia . . . 8

D. Arquitectura técnica de la solución **9**

E. Diseño y desarrollo de la solución de IA **12**

E.1. Un asistente con información de referencia . . . 12

E.2. Preparación del conocimiento . . . 12

E.3. Representación y recuperación . . . 13

E.4. Generación, controles y continuidad . . . 13

E.5. Decisión obtenida de la evolución del RAG . . . 13

F. Integración con herramientas DevOps **14**

F.1. Flujo de trabajo . . . 14

F.2. Integración continua . . . 14
F.3. Contenedores y entorno local . . . 14
F.4. Registro y métricas . . . 14
F.5. Despliegue continuo . . . 15
F.6. Documentación . . . 15

G. Uso de modelos preentrenados 16
G.1. Adaptación sin entrenamiento propio . . . 16
G.2. Ruta y condiciones de uso . . . 16
G.3. Cómo se eligió el modelo y cómo leer las mediciones . . . 17
G.4. Latencia y alcance de la decisión . . . 17

H. Evaluación de la solución 18
H.1. Qué se evalúa y cómo . . . 18
H.2. Resultados técnicos . . . 18
H.2.1. Lo que funciona . . . 18
H.2.2. Dónde hay margen de mejora . . . 18
H.2.3. Robustez . . . 19
H.2.4. El tiempo . . . 19
H.2.5. Qué hemos corregido . . . 19
H.3. Resultados funcionales . . . 20
H.4. Cómo se reproduce . . . 20
H.5. Qué podemos afirmar y qué no . . . 20

I. Conclusiones y trabajo futuro 21
I.1. Una referencia predictiva para detectar anomalías . . . 21
I.2. Detección periódica y avisos de anomalías . . . 21

Referencias 21

A. Introducción y resumen ejecutivo

Gastar en la nube es fácil; entender el gasto, no. Las startups, los equipos pequeños y los desarrolladores independientes acumulan facturas de proveedores cloud sin visibilidad real de dónde se va el dinero. Rara vez tienen un equipo de FinOps, la práctica de dar visibilidad y control al coste de la nube, ni tiempo para analizar esas facturas a mano. Economicon nace para cerrar esa brecha: entender en qué se va el dinero de la nube no debería requerir un máster en facturas ni un equipo dedicado.

Esta memoria presenta Economicon, un asistente FinOps para personas que no son expertas en la materia. El alcance de este Trabajo Fin de Máster es Microsoft Azure. Sobre un conjunto de datos públicos de costes de Azure, servido por una interfaz de programación (API) simulada, la solución ingiere y normaliza el gasto, calcula indicadores que permiten actuar, detecta gasto anómalo y responde preguntas en lenguaje natural. Para responder, un agente conversacional usa generación aumentada

por recuperación (RAG): antes de redactar, el sistema recupera los fragmentos de documentación y de datos que vienen al caso.

A.1. Resumen ejecutivo

- **Problema:** falta de visibilidad y de tiempo para analizar el gasto cloud en organizaciones sin función FinOps.
- **Solución:** un asistente que convierte datos de coste de Azure en indicadores claros, alertas de gasto anómalo y respuestas conversacionales con fuentes citadas.
- **Arquitectura:** aplicación web con backend en Python, procesamiento asíncrono de tareas y dos bases de datos, una relacional y otra vectorial, que se apoya en modelos de lenguaje a través de un gateway (un punto de entrada que gestiona el acceso a esos modelos). El detalle está en d.
- **Enfoque de IA:** modelos preentrenados, sin entrenamiento propio. La adaptación al dominio se logra con RAG sobre los datos de coste y la documentación FinOps. Detalle en e y g.
- **DevOps:** integración y despliegue continuos, decisiones documentadas y cambios revisados mediante pull requests. Detalle en f.
- **Alcance del MVP:** el producto mínimo viable, es decir, la versión mínima que demuestra el circuito completo. Solo Azure, con datos públicos y sin conexión a la facturación real de ningún cliente.

Los resultados y sus límites se presentan en h. Lo que la memoria puede afirmar es que el circuito completo funciona de extremo a extremo con datos reproducibles. Lo que no puede afirmar todavía es que la respuesta sea útil para el público objetivo, porque eso requiere una prueba con usuarios reales.

B. Caso de negocio

B.1. La oportunidad

El gasto en la nube crece de forma sostenida, y la disciplina FinOps —la práctica de dar visibilidad y control al coste cloud— se ha convertido en una necesidad. Sin embargo, las herramientas FinOps del mercado están diseñadas para grandes organizaciones: asumen equipos dedicados, integraciones complejas y presupuestos que una startup o un profesional independiente no tienen. El resultado es una audiencia amplia y desatendida que paga de más porque no dispone de las herramientas ni del tiempo para entender su factura.

B.2. Audiencia objetivo

- **Fundadores de startups** que gestionan su propio gasto cloud sin un equipo dedicado.
- **Desarrolladores independientes** que pagan de su bolsillo la infraestructura de sus proyectos.
- **Equipos pequeños** sin presupuesto para herramientas FinOps empresariales.

B.3. Análisis de mercado

Gartner prevé que el gasto mundial de los usuarios finales en nube pública alcance los 723.400 millones de dólares en 2025, frente a los 595.700 millones de 2024 [1]. Según Flexera, que encuestó en 2025 a 759 profesionales cloud, el 84 % sitúa la gestión del gasto entre sus principales retos y estima en un 27 % el gasto en infraestructura y plataformas en la nube (IaaS y PaaS) que se desperdicia [2].

Este problema ha impulsado el mercado de las herramientas FinOps. Sin embargo, el sector ha mantenido una orientación clara hacia las grandes empresas, pese a que el desperdicio es proporcionalmente mayor en las pequeñas, que son precisamente quienes menos medios tienen para corregirlo. Aquí radica la oportunidad de Economicon: startups, empresas pequeñas o profesionales independientes, donde el problema, en porcentaje, es más grave y las soluciones existentes no terminan de encajar.

Los datos según tamaño de organización confirman la paradoja: la tasa de desperdicio y el desperdicio en valor absoluto se mueven en sentidos opuestos a medida que la empresa crece. Ambos tipos de organizaciones tienen problemas distintos que requieren soluciones distintas. Economicon se centra en el primer grupo, el peor cubierto hoy.

El mercado, además, está saturado de herramientas de visualización dirigidas a quien ya sabe de FinOps: paneles que muestran el gasto y esperan que el usuario sepa interpretarlo. En el segmento pequeño no competimos con la solución empresarial, sino con la herramienta nativa y gratuita del propio proveedor, que casi nadie aprovecha porque no la entiende.

Ese hueco, el de la traducción y no el de los datos, es el que Economicon viene a ocupar.

B.4. Justificación de la solución

Diferenciador central

Un agente conversacional que responde «¿en qué se me va el dinero?» con datos y fuentes, en lugar de exigir que el usuario construya la respuesta a partir de un panel.

La combinación de un motor analítico determinista, para los números (indicadores clave, anomalías y previsiones), con una capa conversacional basada en RAG, para la interpretación y la explicación, responde directamente al problema identificado. El motor analítico garantiza que las cifras son exactas y trazables; la capa conversacional elimina la barrera de conocimiento que impide al segmento objetivo aprovechar esas cifras. Se ha priorizado el vertical de Azure para el MVP por disponer de un conjunto de datos público representativo y de un modelo de datos de coste bien documentado, lo que permite validar el circuito completo sin comprometer datos reales de clientes.

B.5. Viabilidad

Técnica

El circuito completo, desde los datos de coste hasta el asistente, está construido y cubierto por pruebas automáticas: ingesta y normalización del gasto de Azure, indicadores de coste, presupuestos con umbrales, un panel de salud del sistema y un asistente conversacional que responde con un modelo de lenguaje a partir de los fragmentos recuperados y cita la fuente de cada respuesta. La viabilidad técnica se demuestra de extremo a extremo con datos reproducibles, sin depender de la cuenta real de ningún cliente.

Operativa

En local, un único comando de Docker Compose, una herramienta que arranca varios servicios a la vez, levanta doce servicios con proveedores simulados, y catorce al activar la IA. Se alimenta de un conjunto de datos público de Microsoft (FinOps Toolkit v14, con licencia MIT) servido por una API de Azure simulada, de modo que no necesita conectarse a una cuenta real.

Económica

La viabilidad económica se analiza con un caso de negocio sencillo: cuántos clientes necesita Economicon para cubrir sus costes. Se cobra un pago único de despliegue de 200 € por cliente y una mensualidad de 50 €. Los costes son la infraestructura, 1.200 € al mes y dimensionada para hasta 100 clientes, y el desarrollo, 80 horas al mes de mantenimiento y evolución a 25 € la hora; suman 3.200 € al mes. Se comparan dos escenarios a un año: 20 clientes y 100 clientes.

Cuadro B.1. Caso de negocio de Economicon en un año, en euros

Concepto	Conservador (20 clientes)	Optimista (100 clientes)
Ingresos por despliegue (200 € por cliente)	4.000	20.000
Ingresos por mensualidad (50 € durante 12 meses)	12.000	60.000
Costes (3.200 € al mes)	38.400	38.400
Resultado	**−22.400**	**41.600**

Con 20 clientes la plataforma no cubre sus costes, y con 100 sí. El punto de equilibrio está en unos 64 clientes con la mensualidad, o en 48 si se cuenta el pago de despliegue del primer año, ambos dentro de la capacidad de la infraestructura. Para el cliente, la mensualidad se compensa a partir de un gasto cloud de unos 550 € al mes: si Economicon le ayuda a recuperar un tercio del 27 % que, según Flexera, se desperdicia, ahorra el 9 % de su gasto [2]. Por debajo de ese gasto, la mensualidad pesa demasiado frente al ahorro posible.

B.6. Impacto esperado

El impacto que persigue Economicon es que una persona sin formación en FinOps obtenga una respuesta trazable a la pregunta de en qué se le va el dinero, sin tener que construirla a partir de un panel. Para comprobarlo se midieron tres indicadores sobre una batería de 28 consultas de referencia: la proporción de respuestas correctas, con un umbral de 0,8; la latencia de respuesta en el percentil 95, es decir, el tiempo que no superan el 95 % de las respuestas, con un objetivo de 10 segundos; y la proporción de cifras de la respuesta que no se pueden rastrear hasta los datos.

La primera medición, hecha con una versión del chat que todavía no usaba un modelo y devolvía una plantilla con los fragmentos recuperados, dio 0 de 20 casos de respuesta aceptados [3].

B.7. Costes y supuestos

Los costes del proyecto se dividen en dos bloques: el de la inteligencia artificial, medido con el modelo actual, y el de la infraestructura donde se ejecuta el sistema, estimado para la arquitectura de referencia en AWS. Se presentan por separado porque el primero está acotado por diseño y el segundo depende de dónde y durante cuánto tiempo se despliegue el entregable.

Coste de la inteligencia artificial

El chat usa el modelo DeepSeek V4.1 Flash a través de OpenRouter, un encaminador que dirige cada petición a un proveedor de modelos. El coste medido, contrastado con la facturación del proveedor, fue de unos 0,0006 USD por consulta, es decir, unos 0,06 USD por cada 100 consultas [4]. Las representaciones vectoriales de los textos (embeddings), de 1536 dimensiones, cuestan 0,02 USD por millón de unidades de texto [5]. La decisión de arquitectura, aceptada el 4 de octubre de 2026,

fija un techo de desarrollo de 10 USD al mes mediante claves de acceso revocables [5]. El gasto real acumulado es mínimo: 0,0118 USD entre la medición de referencia de agosto y las comprobaciones posteriores [5], y unos 0,00016 USD la medición de referencia con representaciones reales [3]. Los precios de los modelos cambian durante el día y entre proveedores, con un rango amplio entre puntos de acceso, por lo que se comprueban antes de cualquier gasto real.

Coste de infraestructura

En local, el sistema se compone de doce servicios con proveedores simulados y catorce al activar la IA, entre ellos las bases de datos relacional y vectorial, la cola de mensajes y la monitorización. La referencia en AWS usa servicios gestionados equivalentes y no está desplegada. Para esa arquitectura de referencia, con redundancia y preparada para hasta 100 clientes, se estimó el 22 de octubre de 2026 un coste de unos 1.200 € al mes, a 730 horas al mes y sin IVA ni capa gratuita. La IA se estima aparte, en unos 3 USD al mes con modelos de Amazon; no son mediciones ni tarifas de DeepSeek ni de OpenRouter, y no se incluye en el caso de negocio por ser despreciable.

B.8. Riesgos

Los riesgos del proyecto se agrupan en tres bloques: los técnicos, los económicos y los de contexto.

Riesgos técnicos

El principal fue que el asistente no alcanzara la calidad necesaria. La línea base sin modelo, descrita en el impacto esperado, dio el punto de partida: ninguno de los 20 casos evaluados fue aceptado [3]. La probabilidad y el impacto eran altos, y se controló midiendo siempre sobre la misma batería de consultas de referencia y con los mismos indicadores, de modo que la mejora pudiera demostrarse y no solo afirmarse. Ligado a este estaba el riesgo de latencia: las mediciones de agosto, de 11,73 s y 10,88 s, quedaban por encima del objetivo de 10 s. Su probabilidad e impacto eran medios, y se acotó con límites de unidades de texto y un tiempo de espera de 30 s.

Riesgos económicos

El uso de modelos de pago podría generar gasto no previsto, tanto por cambios de precio o de modelo del proveedor como por un uso mayor del esperado. Se consideró de probabilidad media e impacto bajo, y se controló con un tope de gasto en cada clave de acceso, con un techo de desarrollo de 10 USD al mes fijado en la decisión de arquitectura y con la comprobación de precios antes de cualquier gasto real [5]. El gasto real acumulado se mantuvo en céntimos [5]. El segundo riesgo económico fue el coste de la infraestructura de despliegue, que depende del proveedor y del tiempo que el entorno permanezca encendido. Se mitigó estimándolo con los precios públicos del proveedor antes de desplegar y manteniendo el entorno levantado solo el tiempo necesario.

Riesgos de contexto

El segmento objetivo, las organizaciones pequeñas, es una hipótesis de partida que no se contrastó con usuarios. Su probabilidad era media y su impacto alto, y se controló presentándolo siempre como hipótesis y dejando el contraste con usuarios como trabajo futuro. Por último, el plazo: la congelación de la memoria estaba prevista para el 16 de octubre y la entrega para el 23, sujeta a confirmación institucional. La probabilidad era media y el impacto alto, y se controló priorizando el contenido obligatorio, trabajando en pequeñas entregas revisables por el equipo y señalando de forma explícita lo que quedó pendiente.

C. Contribución individual

Economicon organiza la colaboración mediante cuatro funciones rotatorias: liderazgo, pairing/coautoría, revisión de PR y validación. Trello conserva la asignación prevista y GitHub registra acciones enlazadas. Son evidencias complementarias: una asignación no acredita que se haya ejercido el rol, y abrir una PR no equivale a realizar en solitario toda la entrega. [6]

C.1. Roles rotatorios

El corte del 20/10/2026, 20:25–20:26 UTC, comprende 117 historias no archivadas y 143 PR distintas asociadas por JUP en título o rama (147 asociaciones). Incluye historias posteriores al MVP y conserva las 100 del corte anterior. La tabla cuenta asignaciones reconocidas, no participación efectiva. [7]

Cuadro C.1. Asignaciones de rol por persona

Persona	Liderazgo	Pairing	Revisión	Validación	Total
Alejandro Aguado	26	30	22	31	109
Víctor Méndez	28	22	27	28	105
Lucía Mateo	30	28	30	22	110
Paris Arcos	23	26	28	28	105

Se reconocen 429 posiciones de rol y quedan 39 sin resolver de 468 posibles: 10 de liderazgo, 11 de pairing, 10 de revisión y 8 de validación. Incluyen ambigüedades de extracción, dispensas y líneas sin responsable reconocible; no se reparten por equilibrio estadístico. JUP-055 y JUP-127, por ejemplo, sí nombran personas, pero etiquetas repetidas con enlaces impiden resolver automáticamente algunas posiciones. Tampoco todas las tarjetas asignan cuatro personas distintas: siete repiten una identidad entre funciones. [7]

C.2. Quién entregó qué

Las PR integradas son las abiertas por cada persona y fusionadas en develop hasta el 19/10, contrastadas el 20/10: 119 en total, incluidos distintos métodos de integración. Las reviews corresponden al corte del 20/10 en las 143 PR asociadas: tituladas/total de eventos en PR ajenas al autor, contados una vez por PR. Se excluyen PENDING y las intervenciones del autor en su propia PR; se conservan dictámenes históricos, incluso sobre SHA anteriores. Un evento no acredita una validación vigente ni mide esfuerzo. [7]

Cuadro C.2. Integración y revisiones por persona

Persona	PR integradas	Reviews tituladas/total
Alejandro Aguado	56	46/54
Víctor Méndez	17	76/87
Lucía Mateo	33	83/103
Paris Arcos	13	67/91

Alejandro Aguado: normalización de costes Azure (JUP-013), chunking y embeddings (JUP-020), prompt y guardrails (JUP-024), contrato de herramientas del agente (JUP-084), contenedores y CD (JUP-049/052). Implementó el recolector y la documentación inicial de JUP-064; su autoría material se distingue de la apertura de PR76. [6]

Víctor Méndez: inventario y migración del frontend y reconciliación de su capa de datos (JUP-090–095/097), verificación del workspace y cierre de la migración (JUP-103/105), además de liderazgo y contraste del registro JUP-064 y apertura de PR76. [6]

Lucía Mateo: logging y seguimiento de ingestas (JUP-042–045), entorno Docker Compose (JUP-050), recuperación semántica (JUP-022), anomalías (JUP-030/114) y aportaciones de evaluación e interfaz identificadas en JUP-067/070/071/106/107/119/112/117/100. En JUP-064 realizó la revisión de código, con los límites de alcance indicados abajo. [6]

Paris Arcos: ingesta documental y OCR (JUP-124), chunking documental (JUP-125), embeddings y recuperación híbrida (JUP-127), integración de LiteLLM/OpenRouter y servicios asociados (JUP-023/108), KPIs y dashboard (JUP-026/055), además de componentes y pruebas de JUP-047/085/086/051/053. En JUP-064 realizó la validación documentada. [6]

C.3. Trazabilidad y límites de la evidencia

Cada contribución se relaciona mediante JUP, tarjeta, cambios y PR. El flujo previsto exige revisión y validación; el historial conserva excepciones y ausencias de evidencia, por lo que no se afirma que todas las PR hayan recibido dos dictámenes independientes. Por ejemplo, PR1 está integrada sin reviews en la API. La existencia de documentación, pruebas y checks de CI acredita artefactos del equipo, no por sí sola su ejecución por una persona. [6]

Las entregas JUP-017/037/054/068/101/102/111 incorporan excepciones de cierre por plazo y ejecución asistida documentadas; no se atribuyen pairing ni aprobaciones humanas dispensadas. Integración y cierre en Trello son distintos: en el corte del 20/10, JUP-037/101/102 figuran en Backlog y las otras cuatro en Hecho. En JUP-102 la descripción atribuye liderazgo y comprobación propia a Alejandro y dispensa pairing y revisión humanos independientes. [6, 7]

El snapshot histórico del 10/10 reconoce cinco commits distintos con coautoría declarada; esta cifra no se ha actualizado con el corte de roles y reviews del 20/10. Los trailers no demuestran una sesión de pairing y las notas de colaboración no importadas no se reconstruyen. Su ausencia en el registro tampoco demuestra ausencia de trabajo. [6]

PR76 se integró con revisión de Lucía y validación de Paris del 08/10: la primera excluía la recogida real y la segunda tenía alcance offline. PR109 actualizó los datos por excepción de plazo, sin nuevas reviews independientes. Las aprobaciones de PR76 no se trasladan a PR109 ni acreditan conformidad con las filas de esta memoria. Esa conformidad permanece pendiente para los cuatro miembros. [6]

D. Arquitectura técnica de la solución

Versión de arquitectura: 10/10/2026. Base funcional de la versión integrada; documentación y referencia AWS/Terraform de la PR 92. **[Pendiente]** Revisión de Lucía y validación posterior. [8-10]

Componentes. Economicon separa la interfaz React, la API FastAPI en Python y un processor que ejecuta trabajos asíncronos mediante RabbitMQ. La API concentra autenticación, autorización y consultas; el processor se ocupa de la ingesta documental. Esta separación evita mantener peticiones HTTP abiertas durante el procesamiento y permite dimensionar ambos componentes independientemente. Los costes proceden de una API Azure simulada: no se acredita conexión con facturación real. El código se organiza en un monorepo con apps/frontend, apps/backend y apps/processor; infra reúne la configuración de infraestructura, docs las decisiones y openspec los requisitos.

Persistencia. CockroachDB conserva usuarios, tenants, membresías, trabajos, conversaciones y costes. PostgreSQL con pgvector almacena documentos, fragmentos y embeddings. Esta separación mantiene responsabilidades y contratos explícitos; no demuestra por sí misma una ventaja de rendimiento. Las migraciones respetan el propietario de cada tabla y un único ejecutor. La persistencia y la publicación en RabbitMQ no forman una transacción distribuida: confirmar la publicación no equivale a procesamiento exactamente una vez.

Consultas. El navegador envía la pregunta con el token y el tenant seleccionado. La API comprueba identidad, membresía y propiedad de la conversación, persiste la pregunta, obtiene su embedding y recupera fragmentos por similitud coseno, filtrando por tenant. La respuesta incluye referencias. En el corte documentado se construye mediante una plantilla determinista; no se ha acreditado generación conversacional mediante un LLM. El chat es síncrono y no pasa por la cola del processor. Los errores se comunican explícitamente. [8]

Ingestas. La ingesta documental recibe text_content, registra un trabajo y lo publica en RabbitMQ. El processor normaliza, fragmenta, analiza, genera embeddings y almacena el resultado. artifact_uri es metadato, no una descarga automática. La ingesta Azure utiliza un comando independiente contra el simulador. Las consultas de facturación agregan registros completados por intervalo y moneda, rechazan solapamientos y no presentan ahorro realizado sin evidencia. Los paneles que utilizan fixtures tampoco acreditan análisis automático de costes reales.

Inteligencia artificial. El perfil opcional incorpora LiteLLM como gateway hacia OpenRouter, con claves virtuales por consumidor y credenciales de proveedor fuera del navegador. El backend utiliza embeddings y el processor puede solicitar análisis estructurado. Estos contratos no prueban que el chat ejecute un flujo generativo completo ni herramientas FinOps. La autorización permanece en la aplicación. **[Pendiente]** La integración conversacional generativa y su evaluación siguen pendientes de acreditación; las pruebas simuladas no demuestran calidad del modelo.

Seguridad. La autorización combina JWT, membresía y tenant: enviar X-Tenant-Id no concede acceso por sí solo. Las conversaciones se restringen a su propietario y los documentos al tenant autorizado; las consultas vectoriales preservan ese aislamiento. El entorno de demostración no acredita MFA, revocación completa ni endurecimiento para exposición pública. Los secretos se suministran mediante configuración protegida y no forman parte del frontend ni de los entregables documentales.

Reproducibilidad. Docker Compose define nueve servicios y cinco volúmenes en el perfil base; el perfil de IA añade dos servicios y un volumen. Permite reproducir dependencias y ejecutar la demostración con datos simulados. Los puertos publicados, CockroachDB en modo inseguro y el servidor de previsualización frontend requieren controles adicionales para otro contexto operativo. El entorno local es la referencia implementada, sin acreditar un despliegue productivo AWS.

Observabilidad y entrega. Prometheus recoge métricas de backend y processor y Grafana las visualiza. Los logs JSON y los identificadores de petición ayudan a correlacionar eventos, sin equivaler a trazabili-

dad distribuida completa; tampoco se acredita un receptor externo de alertas. GitHub Actions verifica código y documentación mediante CI. El despliegue continuo corresponde a JUP-052 y requiere su propia evidencia. Los procedimientos de pruebas y los resultados se desarrollan en los apartados f y h para evitar duplicidades.

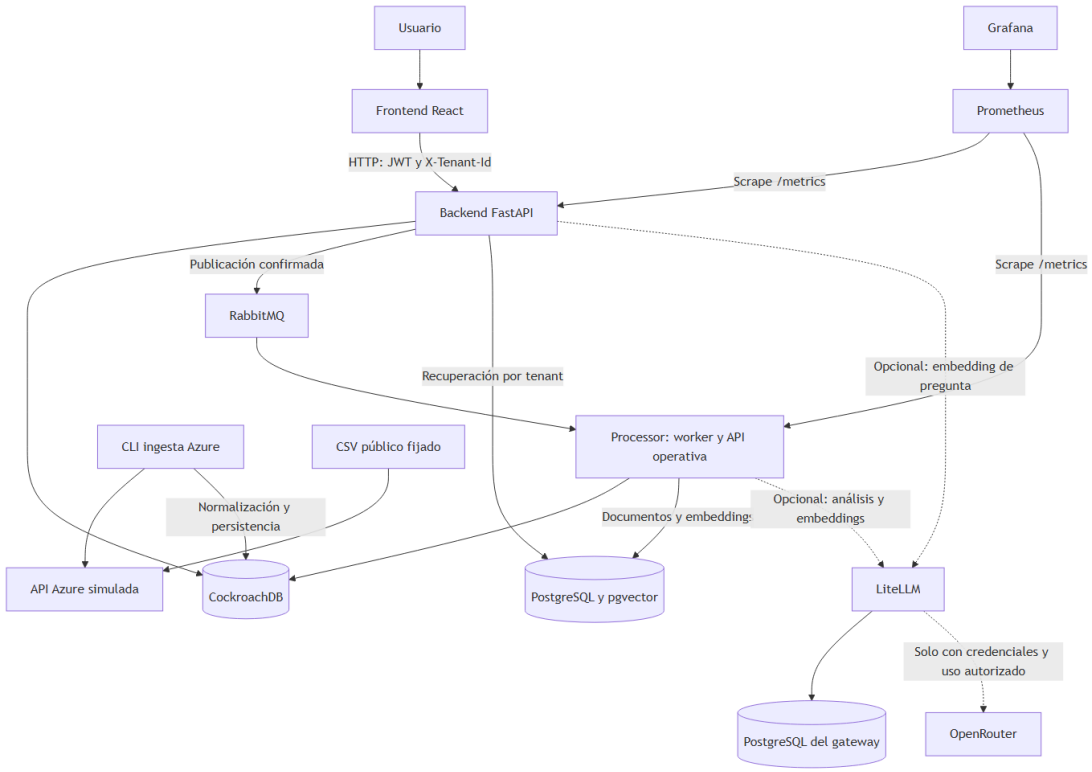


Figura D.1. Componentes del corte funcional; líneas discontinuas: integraciones opcionales. [8]

Alternativa documental AWS. El frontend se alojaría en S3 privado y CloudFront, con WAF y ACM; la API y el processor se ejecutarían en ECS Fargate, con un ALB interno para la API y subredes privadas. Amazon MQ conservaría RabbitMQ y RDS PostgreSQL alojaría datos operativos y vectores. Cognito, Bedrock, Secrets Manager, IAM/KMS y CloudWatch cubrirían identidad, IA, secretos, permisos, cifrado y observabilidad. El esquema AWS detalla las dependencias. Sustituir CockroachDB, la autenticación y los proveedores actuales exige adaptar SQL/migraciones, tokens y modelos/embeddings; no basta cambiar variables. Son brechas hipotéticas de compatibilidad, no tareas de despliegue pendientes de este encargo.

Terraform y límites. La plantilla Terraform complementa el diseño y define recursos de red, aplicación, datos y seguridad. Se validó localmente mediante formato, validación estática, seis pruebas con proveedores simulados y ocho casos de enrutamiento. No acredita un plan real, compatibilidad integral ni operación en AWS. El pipeline CodeConnections–CodePipeline–CodeBuild–ECR–CDK/CloudFormation es conceptual; Terraform es una alternativa de aprovisionamiento y no implementa ese pipeline. El alcance acordado es documental: no se han creado recursos ni realizado despliegues AWS. [10]

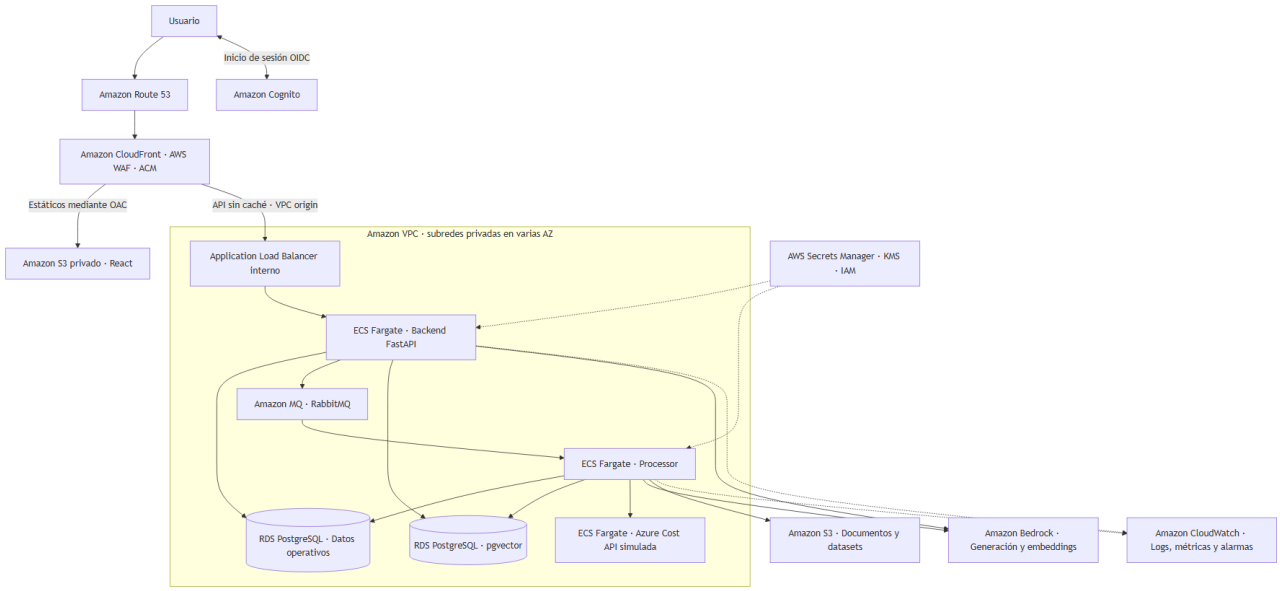


Figura D.2. Referencia conceptual AWS; no representa un despliegue realizado. [9]

Cuadro E.1. Decisiones de preparación y recuperación del conocimiento

Elemento	Elección	Qué significa
Representación del texto	text-embedding-3-small; 1.536 dimensiones	Cada pasaje se representa mediante un vector numérico compatible con el de la consulta. [11, 12]
Fragmentación	Objetivo 600; máximo 800; solapamiento hasta 60 tokens	Un token es una unidad de texto procesada por el modelo; el presupuesto se aplica por fragmento y respeta su estructura. [11]
Recuperación	Distancia coseno ≤ 0,6; hasta 4 pasajes principales	La distancia coseno mide cuánto se parecen dos vectores (0 es idéntico) y solo se admiten los pasajes por debajo de 0,6; los vecinos y el apoyo por palabras tienen reglas propias. [11]
Resultado histórico que apoyó la elección	Configuración híbrida elegida: 68 de 68 respuestas obtenidas	Completitud de una campaña concreta; no equivale a exactitud universal. [13]
Evolución ensayada, pendiente de integración	Hasta 4 intentos; temperaturas 0 / 0,2 / 0,4 / 0,6	La temperatura regula la variabilidad; aumenta en los reintentos de una salida rechazada. [14]

E. Diseño y desarrollo de la solución de IA

Este apartado explica cómo Economicon consigue que sus respuestas se apoyen en información verificable: qué conocimiento prepara, cómo lo recupera y qué controles aplica antes de mostrar una respuesta. La tabla E.1 resume las decisiones que se desarrollan a continuación.

E.1. Un asistente con información de referencia

FinOps relaciona costes, decisiones y responsabilidades. Un modelo de lenguaje puede explicar conceptos generales, pero sus conocimientos previos no contienen necesariamente los datos de una consulta ni las reglas de una organización. Economicon utiliza generación aumentada por recuperación, RAG: selecciona información autorizada y la incorpora a la petición antes de redactar la respuesta. Así, la explicación puede vincularse con sus fuentes en lugar de depender solo de la memoria del modelo. [11]

El recorrido distingue información documental, datos aportados por la persona y evidencia financiera obtenida por el servidor. La pregunta se recibe con la identidad y los permisos de la sesión; se recuperan los pasajes pertinentes y el modelo prepara una respuesta. Después, el servidor comprueba estructura, referencias y operaciones admitidas antes de confirmar el resultado. La figura E.1 muestra también cómo se prepara el conocimiento que alimenta ese ciclo. [11]

El MVP cubre Azure con datos públicos de Microsoft FinOps Toolkit y una API que simula el servicio de consulta de costes. No utiliza la facturación de un cliente real. El espacio de cada organización, o tenant, delimita el acceso a sus documentos y conversaciones; un escenario hipotético permanece identificado como tal y no se convierte en gasto verificado. [11, 15]



Figura E.1. Preparación del conocimiento y ciclo de una respuesta en la referencia integrada. La evolución de reparación y reintento se distingue en la sección E.4.

E.2. Preparación del conocimiento

La ingesta admite texto manual, Markdown, TXT, DOCX, DOC y PDF digital, escaneado o mixto. La extracción y el reconocimiento óptico de caracteres, OCR, se realizan localmente con recursos acotados. MinIO, el almacén de ficheros del sistema, conserva temporalmente los originales; su eliminación sigue a la persistencia del texto normalizado, los vectores y el estado final del trabajo. Una limpieza fallida se recupera sin repetir el procesamiento. [11]

Un chunk o fragmento es la unidad de texto almacenada y recuperada por separado. El algoritmo respeta secciones, listas, tablas y código cuando caben y aplica cortes reproducibles cuando exceden el presupuesto de la tabla E.1. El solapamiento conserva continuidad entre fragmentos y queda registrado; los metadatos permiten reconstruir el texto normalizado retirando repeticiones, sin prometer la maquetación original. [11]

E.3. Representación y recuperación

Un embedding es una representación numérica que permite comparar la proximidad de significado entre textos. PostgreSQL, ampliado con pgvector, funciona como base vectorial: almacena esas representaciones y recupera pasajes por similitud. Documento y consulta deben pertenecer a un espacio compatible, identificado por modelo, proveedor, dimensión, tokenizador y preparación del texto. [11]

Las variantes identifican cambios acumulativos: A empleó fragmentos por caracteres y búsqueda densa, basada en vectores; B introdujo fragmentación estructural; C añadió título y sección a la representación; D incorporó búsqueda léxica, que localiza palabras e identificadores concretos. La combinación complementa la cercanía semántica cuando importan una denominación, magnitud o condición literal. [13]

D fusiona las listas mediante Reciprocal Rank Fusion, RRF: puntúa la posición de cada candidato en los rankings sin tratar sus escalas como equivalentes. La distancia de la tabla E.1 limita la vía densa; el rescate léxico conserva sus condiciones de admisión. El índice se prepara por generaciones completas antes de activarlo y la caché se separa por organización, espacio y texto exacto. Así se evita mezclar vectores incompatibles o atribuir al pasaje citable el contexto añadido a su representación. [11]

E.4. Generación, controles y continuidad

Un prompt versionado es un conjunto de instrucciones identificado para reproducir qué se pide al modelo. La salida estructurada organiza la respuesta en campos con tipos y límites definidos. El servidor valida el contrato y conserva mensajes y fuentes; los cálculos permitidos, unidades y autorizaciones se resuelven mediante código. El resumen de turnos completos y los mensajes recientes acotan la ventana de contexto, y el envío se bloquea durante la compresión. [11]

La evolución evaluada, pendiente de integración, combina rechazo, reparación y reintento. Omite una métrica no verificada con un aviso visible, elimina segmentos de interpretación vacíos y recupera el estado de la conversación tras fallo o cancelación. Reintenta una salida rechazada con la variabilidad creciente indicada en la tabla E.1. Este comportamiento pertenece al candidato medido, no a la versión integrada de referencia. [14]

E.5. Decisión obtenida de la evolución del RAG

Elegimos D por su completitud observada y por seleccionar contexto más útil en las preguntas adicionales y específicas; A y B conservaron mayor cobertura en las originales. Un juez LLM, modelo de lenguaje que valoró el contenido guardado, apoyó esa lectura. Se trata de una decisión ligada al corpus, código y muestra históricos, sin superioridad general demostrada. El detalle y sus denominadores se conservan en el informe de campaña; los controles estructurales no sustituyen la evaluación de pertinencia de la respuesta. [13]

F. Integración con herramientas DevOps

Este apartado describe cómo se construye, se prueba y se despliega Economicon de forma automatizada, y qué evidencia deja cada paso. Recorre el flujo de trabajo, la integración continua, los contenedores, el registro y las métricas, el despliegue continuo y la documentación.

F.1. Flujo de trabajo

Economicon vincula la ejecución del proyecto mediante un mismo identificador de tarea en Trello, OpenSpec y GitHub. Trello conserva alcance, responsables y estado; OpenSpec define requisitos y escenarios de aceptación en el propio repositorio; GitHub conserva código, documentación, decisiones técnicas y revisiones. Las ramas de tarea proponen cambios a la rama de desarrollo (develop), mientras que la rama principal (main) representa la línea estable. El proceso separa liderazgo, trabajo en pareja (pairing), revisión y validación: una asignación de rol o una comprobación automática no demuestra por sí sola participación ni aceptación humana. [5, 16]

F.2. Integración continua

La integración continua (CI) se ejecuta en GitHub Actions ante un envío de rama (push), una solicitud de cambios (pull request) a develop o main, por activación manual y como flujo reutilizable por el despliegue continuo (CD). En Ubuntu, el flujo prepara Node.js 22 y Python 3.12. Comprueba trazabilidad, higiene del repositorio y OpenSpec; ejecuta las suites de pruebas de backend, processor y API de costes; y verifica el análisis de estilo, las pruebas, la construcción y los tipos del frontend. En Python, compileall comprueba sintaxis y compilación a bytecode, sin sustituir un análisis de estilo. Las dependencias del workspace se instalan con las versiones exactas fijadas en el fichero de bloqueo. [17]

La evidencia de la integración continua recoge una ejecución de pull request del 05/10/2026 con siete jobs correctos y otra de push con los seis técnicos correctos y la política de PR omitida, como corresponde a ese evento. Esos resultados pertenecen a aquella entrega. La CI comprueba estáticamente la topología Docker y prueba las herramientas de diagnóstico; no construye ni arranca el stack completo. Las pruebas que requieren servicios reales y se omiten en una ejecución no acreditan esa integración. Por ello, los ensayos con contenedores se documentan separadamente. [17]

F.3. Contenedores y entorno local

Cuatro Dockerfiles empaquetan frontend, backend, processor y API de costes. Docker Compose conecta las aplicaciones con sus bases de datos, RabbitMQ, Prometheus y Grafana. Las aplicaciones cuentan con comprobaciones de salud (healthchecks), usuario sin privilegios (no root) y controles de ejecución definidos en Compose; sus imágenes base están fijadas por huella (digest). Este endurecimiento no se extiende de forma idéntica a todos los servicios auxiliares. local:doctor comprueba la preparación del entorno y local:smoke, una prueba de humo, recorre salud, autenticación demo, ingesta y procesamiento. Un primer ensayo documenta la construcción y la prueba de humo en DockerServer; un segundo acredita arranque, prueba de humo, un control negativo con el processor detenido y persistencia tras reiniciar en Docker Desktop. Son entornos de ensayo, con datos de prueba y proveedores simulados, no una certificación de producción. [18]

F.4. Registro y métricas

El registro de backend y processor emite eventos JSON a la salida estándar con fecha, servicio, nivel y contexto. Cada petición recibe un identificador (request_id). En POST /jobs/ingest, el backend lo transmite en el mensaje de RabbitMQ; el worker conserva el identificador válido o genera otro si falta o

es inválido. Así puede seguirse el recorrido de una ingesta asíncrona y sus errores. Los eventos HTTP registran método, plantilla de ruta, estado y duración, y se redactan campos sensibles. Las pruebas y su evidencia verifican esa correlación, incluidos reintentos; no acreditan trazas distribuidas completas ni cobertura de todos los procesos batch. [19]

Prometheus consulta los endpoints /metrics de backend y processor cada 15 segundos. Los contadores e histogramas alimentan un panel provisionado en Grafana con la latencia p95 (el tiempo que no supera el 95 % de las peticiones), los errores 5xx (fallos del servidor), las ingestas, las consultas y las alertas. La regla detecta un incremento superior a dos marcas de fallo persistidas en una ventana móvil de cinco minutos y exige que la condición se mantenga dos minutos: cuenta también reintentos, no trabajos únicos. La evidencia de alertas registra la transición hasta el estado de disparo (Firing); una prueba posterior usó un exportador de métricas sintético. Esta instrumentación permite diagnóstico técnico, pero no constituye un objetivo de nivel de servicio (SLO) demostrado ni una evaluación de calidad de las respuestas. No se acredita un receptor externo de alertas. [20]

F.5. Despliegue continuo

El despliegue continuo privado se integró en develop el 11/10/2026 mediante una pull request. GitHub Actions reutiliza la CI para declarar elegible una versión concreta del código (SHA); un agente pull en DockerServer descarga esa revisión, prepara una release aislada, comprueba disponibilidad y ejecuta cinco verificaciones funcionales antes de promoverla. El diseño incluye bloqueo de concurrencia, puertos accesibles solo desde la propia máquina (loopback), secretos locales y recuperación. El entorno es de desarrollo y validación, con datos desechables y proveedores simulados. El rollback reactiva la release anterior con sus propios volúmenes; no revierte migraciones ni recupera los datos de la release retirada. [21]

La primera promoción automática integrada quedó acreditada el 11/10: la evidencia de cierre registra el mismo enlace a la versión activa (current) y el mismo identificador de ejecución (run_id), cinco pruebas de humo de cinco y nueve contenedores en marcha con once puertos publicados solo en loopback. Una lectura posterior del estado y del registro del sistema confirma una segunda promoción, cuya release anterior es la primera. El ensayo local anterior, sin identificador de ejecución (run_id=null), se conserva como evidencia histórica distinta. Las cinco comprobaciones recorren salud, login, ingesta simulada, facturación y job RabbitMQ. **[Pendiente]** recuperación tras reiniciar el host y validación integral de producción. La promoción simulada no demuestra calidad de respuestas ni inferencias reales. [21]

F.6. Documentación

La documentación mantiene el vínculo entre cada decisión, su implementación y sus límites: README y manuales para ejecutar, decisiones de arquitectura (ADR) para justificar alternativas y los ficheros de evidencia para reproducir verificaciones. El registro de decisiones reúne ese vínculo sin convertir automáticamente propuestas en decisiones ratificadas; las decisiones sobre métricas y despliegue continuo conservan el estado «propuesta» en las fuentes consultadas. [5, 16, 20, 21]

Cuadro G.1. Configuración de referencia y evidencias distintas para la selección

Ámbito	Dato o elección	Qué cuenta
Modelo actual (configuración integrada)	DeepSeek V4.1 Flash; DeepInfra, 8 bits; sin cambio automático de ruta	Único generador del ejemplo versionado; no certifica un servicio externo. [12]
Comparativa histórica de contenido	27 de 28 aceptables en cada modelo	GLM-5.2 y Flash: preguntas previstas por modelo; valoración posterior de un juez LLM, un modelo de lenguaje que valora las respuestas. [22]
Comparativa histórica de coste	GLM: 0,1806 USD; Flash: 0,0392 USD	Consultas e intentos de las 28 operaciones por modelo, incluidos embeddings de consulta; excluye indexación inicial. [22]
Medición posterior del modelo actual: latencia, antes / tras el arreglo	p95 total: 41,3 / 42,4 s; objetivo: 10 s	p95: tiempo bajo el que se sitúa el 95 % de las operaciones completadas con latencia total registrada. Banco de 28 casos: n = 28 antes y 27 después. No cuenta respuestas aprobadas por contenido ni cada intento del proveedor. [23, 24]

G. Uso de modelos preentrenados

Economicon no entrena modelos propios: usa modelos preentrenados a través de un gateway y los guía con instrucciones y contexto. Este apartado explica por qué, por qué ruta se accede a ellos, cómo se eligió el modelo y qué se puede afirmar sobre su coste y su tiempo de respuesta. La tabla G.1 resume los datos clave.

G.1. Adaptación sin entrenamiento propio

Un modelo preentrenado ha aprendido patrones antes de integrarse en la aplicación. Las opciones son entrenarlo desde el inicio, ajustar sus pesos mediante fine-tuning o utilizarlo con instrucciones y contexto. Para el MVP se eligió la tercera: concentró el desarrollo en las fuentes y el control de la respuesta, sin mantener un ciclo propio de entrenamiento. La generación de embeddings emplea otro modelo existente y no modifica sus pesos. [11, 12]

G.2. Ruta y condiciones de uso

LiteLLM actúa como gateway de modelos: una entrada común para credenciales, límites y acceso; OpenRouter encamina la petición al proveedor externo. El alias economicon-chat, nombre estable usado por la aplicación, resuelve al único generador de referencia de la tabla G.1. La configuración figura en la versión integrada de referencia; su registro de decisión sigue propuesto y pendiente de ratificación del equipo. [5, 12]

ZDR, Zero Data Retention, identifica la política de retención cero exigida al encaminamiento. La configuración rechaza recopilación y fallback, el cambio automático de ruta ante un fallo; el historial de la aplicación conserva su propia persistencia. La cuantización FP4/FP8 representa parámetros con precisión reducida de cuatro u ocho bits. El presupuesto de tokens, las unidades de texto que procesa el modelo, limita la longitud permitida y se ajusta junto a la calidad, la latencia y el coste. Estas condiciones configuradas no equivalen a una atestación externa de cada inferencia. [12, 22]

G.3. Cómo se eligió el modelo y cómo leer las mediciones

GLM-5.2 fue el modelo principal del chat y DeepSeek V4 Pro una alternativa distinta de V4.1 Flash; ninguno de los dos sigue activo en la configuración de referencia. Para elegir el modelo se compararon GLM-5.2 y Flash con las mismas preguntas, corpus, índice y contexto sintético. El coste comparable de la tabla G.1 apoyó la elección de Flash; GLM necesitó menos reintentos y tuvo tiempos más regulares. La valoración del juez LLM describió el contenido de aquella muestra, no una clasificación general de modelos ni una validación humana. [22]

Las mediciones posteriores por reglas y con una revisora humana única emplearon otros recursos y métodos de captura. Aumentó el número de casos aprobados, pero cambiaron denominadores y puntos juzgados; el veredicto del instrumento siguió siendo «No aceptado». Por ello se conservan ambas lecturas separadas. Las métricas de Grafana son registros técnicos, no juicios semánticos, y los importes recientes del informe de autor siguen pendientes de contraste independiente de sus originales. [23, 24]

G.4. Latencia y alcance de la decisión

El percentil de latencia de la tabla G.1 supera ampliamente el objetivo; el arreglo no lo alcanza. Es tiempo del recorrido observado, con aplicación, cola y validación, no solo de generación del modelo. La recuperación tampoco admite una comparación directa entre cortes, porque cambió cómo se capturó. [23, 24]

Acotar la salida sin truncarla, estudiar otra ruta compatible con la política de privacidad o reutilizar contexto mediante caché son vías de ajuste no evaluadas aquí. La elección mantiene una ventaja de coste observada en la comparación histórica y un margen de mejora de tiempo y calidad; no convierte las mediciones de candidatos pendientes de integración en garantías del sistema desplegado. [22-24]

H. Evaluación de la solución

Un asistente que explica costes solo es útil si acierta, cita de dónde saca las cifras, responde en un tiempo razonable, cuesta poco y se comporta bien con datos incompletos. Este apartado mide esas cinco cosas sobre el sistema actual, con DeepSeek V4.1 Flash como único modelo del chat.

H.1. Qué se evalúa y cómo

Un asistente con RAG puede fallar al recuperar, al generar o al citar, y por eso se mide cada etapa. El asistente puede fallar al recuperar, al redactar o al citar, así que medimos cada etapa, usando un banco de preguntas que tiene 28 items.

El banco tiene 28 preguntas: 20 de responder (tienen respuesta en los datos), 6 de pedir el dato que falta y 2 de abstenerse. Cada pregunta es un caso y se aprueba solo si cumple todas sus comprobaciones. Las objetivas son reglas automáticas (que aparezca la cifra correcta, que no se afirme algo prohibido); las juzgadas las decide una persona (por ejemplo, que se pida el dato que falta) [3]. La robustez se mide con 29 preguntas adicionales, 13 del banco anterior con datos intactos y 16 con un defecto introducido a propósito: etiquetas, reparto de costes compartidos, indicadores de coste, ámbito de los datos y anomalías

Los umbrales de admisibilidad son cuatro: el 80 % de las preguntas de responder aprobadas (propuesta pendiente de validar por el equipo) [3], el 90 % de las comprobaciones objetivas cumplidas, ninguna cifra sin rastro en casos críticos y un percentil 95 de latencia de 10 s o menos, es decir, que el 95 % de las conversaciones no supere ese tiempo. Los tres últimos se fijaron en la decisión de arquitectura sobre la pasarela de modelos, el servicio que reparte las peticiones entre modelos y controla su gasto [5].

H.2. Resultados técnicos

Se midió el banco en tres cortes: la plantilla sin modelo (la primera versión del chat) y DeepSeek V4.1 Flash antes y después del arreglo de cortes de respuesta. [3, 24]

Lo que funciona

El chat atendió todas las peticiones del banco sin errores del servidor. Cuando cita, cita bien: todas las citas del último corte apuntaban a un fragmento realmente recuperado y todas las respuestas cumplieron el esquema [24]. Además es barato. Contrastamos las consultas de la última ventana de medición con la factura del proveedor de acceso a modelos (OpenRouter): el gasto estimado y el facturado difirieron en menos de un 1 % [4]. Con DeepSeek V4.1 Flash, 100 consultas al modelo cuestan unos 0,06 USD, frente a unos 0,26 USD con el modelo anterior; son ventanas y recursos distintos, y no atribuimos la diferencia solo al modelo.

Dónde hay margen de mejora

El acierto de las preguntas de responder y el de las comprobaciones objetivas están todavía por debajo de los objetivos, y por eso la herramienta de puntuación aún no da el resultado por aceptado en ninguno de los tres cortes. Las cifras admiten tres lecturas. La más exigente es la de casos completos, que es la de la tabla. La segunda mira las comprobaciones sueltas y es más indulgente, porque un solo fallo no descarta el resto: en las juzgadas el cumplimiento es mucho mayor. La tercera es la de un juez LLM, que en la comparativa entre modelos dio casi todas las respuestas por aceptables (27 de 28) [22]; mide otra cosa y no es comparable. Además, parte de los fallos es del lector numérico automático, que

Cuadro H.1. Resultados técnicos del chat en tres cortes

Medida	Primer corte	Antes del arreglo [24, 25]	Tras el arreglo [24, 26]	Objetivo
Banco de 28 preguntas				
Casos aprobados, de 28	0	10	13 (14 fallan, 1 bloqueado)	—
Preguntas de responder aprobadas	0 de 20	6 de 20	7 de 19	80 %: no se cumple
Comprobaciones objetivas cumplidas	26 de 51	31 de 51 (60,8 %)	28 de 46 (60,9 %)	90 %: no se cumple
Comprobaciones juzgadas cumplidas	—	25 de 30 (83,3 %)	26 de 28 (92,9 %)	—
Cifras sin rastro	—	1 (no crítico)	0	0 en críticos: se cumple
Percentil 95 de latencia	—	41,3 s	42,4 s	10 s: no se cumple
Robustez (29 casos)				
Aprobados / fallidos / bloqueados	7 / 17 / 5	7 / 21 / 1	15 / 10 / 4	—
Aprobados sobre evaluados	29,2 %	25,0 %	60,0 %	—
Prohibiciones incumplidas	0 de 55	0 de 64	0 de 59	—

comprueba que las cifras de la respuesta coincidan con las esperadas: antes del arreglo, la mitad de los casos fallidos fallaba solo por eso y tenía las cuentas correctas a mano [23]; después no lo hemos cuantificado **[Pendiente]**. Con el mismo método de captura, la recuperación localizó el documento correcto en todos los casos de responder y la sección correcta en la gran mayoría [24]; las campañas RAG A–D se resumen en [13].

Robustez

El asistente debe detectar un defecto en los datos y decirlo, en lugar de inventar un resultado. Tras el arreglo aprobó todos los casos de reparto de costes compartidos y de ámbito de los datos, pero solo una parte de los de etiquetas defectuosas y de indicadores de coste, y el de anomalías quedó bloqueado. En ningún corte afirmó algo prohibido. La mayoría de los fallos del último corte lo son solo de la comprobación numérica automática y no se han revisado a mano **[Pendiente]**. [26]

El tiempo

La latencia es el otro margen de mejora: el percentil 95 está en torno a cuatro veces por encima del objetivo, la mediana ronda los 20 s y el máximo supera el minuto [24]. La llamada al modelo tardó entre 4 y 16 s; el resto es servidor, cola y validador. Al ser una medida del ordenador local, queda pendiente repetirla en la infraestructura definitiva.

Qué hemos corregido

Con preguntas sobre cifras de pantalla, una sola regla incumplida rechazaba la respuesta entera aunque el modelo hubiera acertado. El arreglo reintenta dentro de los cuatro intentos con temperatura creciente

(regula cuánto varía el modelo), omite la cifra no verificada conservando el resto de la respuesta con un aviso, y recupera la conversación tras un fallo. En una batería propia de 14 preguntas repetidas 4 veces, las operaciones fallidas pasaron de más de un tercio a una sola de 56 [14]. El aviso de cifra omitida ya se muestra en la interfaz, en el panel y en el historial [14], y siguen sin resolverse las frases con huecos donde iría una cifra.

H.3. Resultados funcionales

El detector de anomalías usa reglas deterministas, sin modelo. Con 7 eventos inyectados en datos sintéticos detectó 5: precisión de 5 de 5 (todo lo que marcó era un evento real) y recall del 71,4 % (la parte de los eventos que encontró), que sube al 100 % sobre los 5 detectables por diseño. El retraso fue de 0 días por construcción. No detecta un pico desde cero ni la primera aparición de un servicio, porque exige una semana de referencia con coste positivo [27]. En los dos tenants de demostración (clientes con sus propios datos), la frescura fue de 0 días y la reconciliación con la fuente dio diferencia 0 en los tres pares tenant-divisa; al ser la fuente sintética, confirma la coherencia interna y no la integridad de datos reales [27]. Las métricas de negocio (tiempo ahorrado, cobertura de gasto asignado, ahorro potencial) tienen definición y un ejemplo sintético reproducible; el piloto con usuarios y el beneficio real no se han medido.

H.4. Cómo se reproduce

Las medidas se cargan en un panel de Grafana (herramienta de monitorización) que registra el modelo servido en cada ejecución, y cada tabla se regenera con un comando del repositorio (anexo). [5]

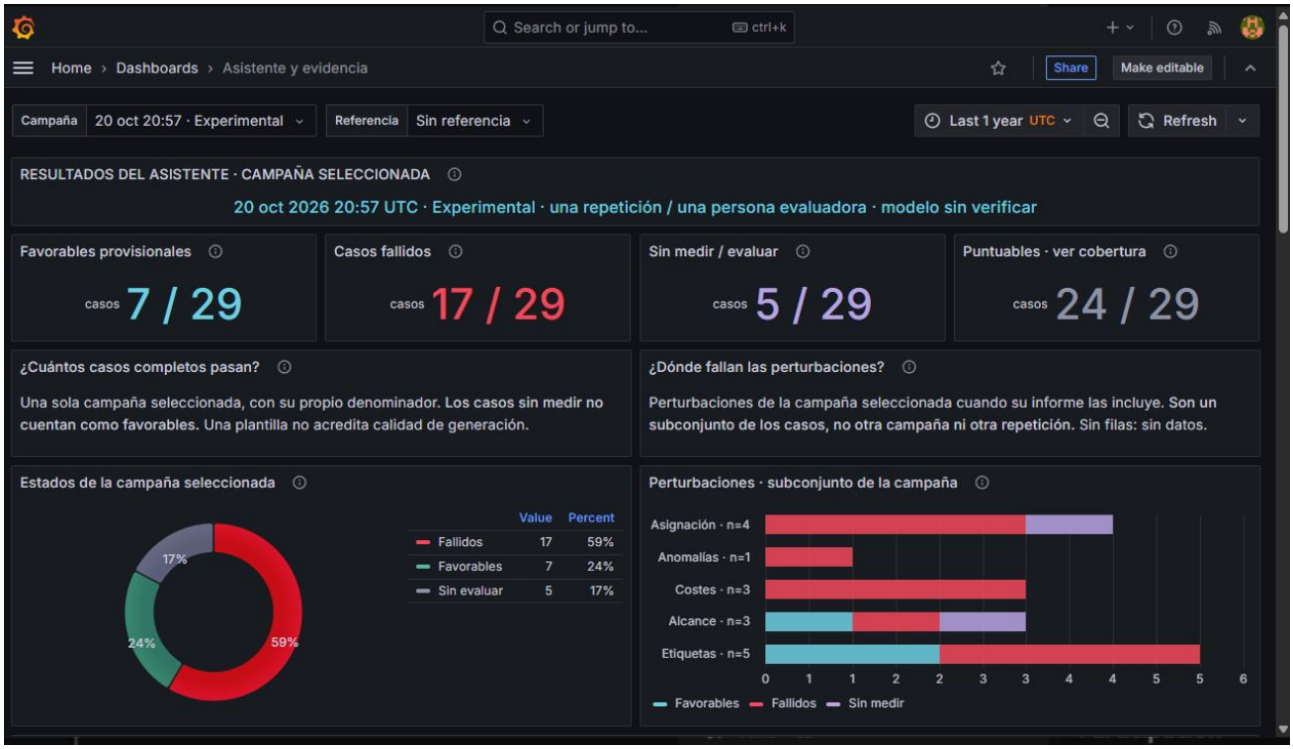


Figura H.1. Dashboard provisional en Grafana

H.5. Qué podemos afirmar y qué no

Con 28 preguntas, una repetición y una revisora afirmamos que el sistema funciona de extremo a extremo, que cita con corrección y que su coste es bajo. No afirmamos la superioridad de un modelo,

que el arreglo mejore la calidad en general ni la calidad en uso real, porque todo se midió en un ordenador local, con datos sintéticos, un corpus pequeño y casos críticos con un único decisor [24]. La identidad del modelo es la que declara la configuración, y la batería de conformidad por requisito no se repitió con el modelo actual.

I. Conclusiones y trabajo futuro

I.1. Una referencia predictiva para detectar anomalías

Actualmente, una anomalía de coste se detecta comparando el gasto de los últimos siete días con el de los siete anteriores: si sube más de un 20 % y más de 100 unidades monetarias diarias, se marca como candidata. Se optó por esta regla determinista porque siempre da el mismo resultado y cada alerta puede explicarse.

Su limitación es que toma como referencia la semana anterior, por lo que no distingue un crecimiento normal de un salto inesperado. Como trabajo futuro se propone sustituirla por un modelo predictivo que estime el gasto esperado de cada día y compare el real con esa estimación. La regresión lineal sería un punto de partida sencillo, aunque modelos con estacionalidad o técnicas no supervisadas podrían ajustarse mejor; la elección dependería de compararlos con datos reales, valorando tanto su precisión como su explicabilidad. Para ello haría falta un histórico de varias semanas o meses, del que hoy no se dispone.

I.2. Detección periódica y avisos de anomalías

En la versión entregada, las anomalías de coste se detectan cuando la persona usuaria pulsa «Detectar ahora». Cada una se guarda como un episodio por cliente, moneda y servicio, con su criticidad y un estado que se puede seguir. El siguiente paso es detectarlas de forma periódica y avisar con tiempo antes del cierre mensual. Quedó fuera por falta de calendario, y porque la ingesta de costes también se lanza a mano, de modo que una detección automática solo vigilaría lo ya cargado. La propuesta es un servicio propio, apagado por defecto, que ejecute la detección existente y registre su resultado, con aviso primero dentro de la aplicación y después, si procede, mediante una regla de Grafana.

Referencias

[1] Gartner. *Gartner Forecasts Worldwide Public Cloud End-User Spending to Total $723 Billion in 2025*. 19 de nov. de 2024. URL: https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025.

[2] Flexera. *State of the Cloud Report 2025*. 2025. URL: https://info.flexera.com/CM-REPORT-State-of-the-Cloud-AWS.

[3] *Evaluación de las respuestas del chat (JUP-070): reglas, criterios y evidencia*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/ded0b4346a4e11442448df980247c01f4367c867/docs/validation/JUP-070-evaluation.md.

[4] *Coste de IA por consulta y contraste con la facturación del proveedor (JUP-133)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/ded0b4346a4e11442448df980247c01f4367c867/docs/evidence/JUP-133/JUP-133-ai-cost.md.

[5] *Registro de decisiones de arquitectura (ADR-0002, ADR-0022 y ADR-0026)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/ded0b4346a4e11442448df980247c01f4367c867/docs/adr.

[6] *Guía y registro de contribuciones del equipo*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/ded0b4346a4e11442448df980247c01f4367c867/docs/contributions.

[7] *Tarjeta JUP-064: contribuciones del equipo y corte del 20/10/2026*. 20 de oct. de 2026. URL: https://trello.com/c/wluz6AGW.

[8] *Arquitectura técnica de la solución*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/d523bc393b0be78674e540a242f3bc9cbeebc8f7/docs/architecture.md.

[9] *Arquitectura de referencia en AWS*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/d523bc393b0be78674e540a242f3bc9cbeebc8f7/docs/architecture-aws.md.

[10] *Plantilla Terraform de referencia y validación*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/d523bc393b0be78674e540a242f3bc9cbeebc8f7/infra/aws-reference/README.md.

[11] *Especificaciones OpenSpec del asistente (conversación, ingesta, fragmentación y recuperación)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/4567e8e7667c3fa5ad2d17eedd2f3da7a332e4bb/openspec/specs.

[12] *Configuración de referencia del gateway de modelos*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/4567e8e7667c3fa5ad2d17eedd2f3da7a332e4bb/infra/litellm/config.example.yaml.

[13] *Informe de campañas RAG A–E*. Documento en Google Drive. 17 de oct. de 2026. URL: https://drive.google.com/file/d/12hQ7bGLJ17aVbHofBy1Rikg4DMlR8eV5/view.

[14] *Medición de los cortes de respuesta del chat*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/5ba296252b2455fb9c285ed0d024bb93035bed99/docs/evidence/chat-cortes-de-respuesta-medicion.md.

[15] *Conjunto de datos de Azure: fuente y licencia*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/4567e8e7667c3fa5ad2d17eedd2f3da7a332e4bb/docs/data/azure-sample-dataset.md.

[16] *Gobernanza del proyecto*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/CONTRIBUTING.md.

[17] *Integración continua y evidencia de validación (JUP-051)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/.github/workflows/ci.yml.

[18] *Docker Compose y ensayos en Docker (JUP-049 y JUP-050)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/docker-compose.yml.

[19] *Convenciones de los servicios Python y verificación de trazas (JUP-044)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/docs/manuals/python-service-conventions.md.

[20] *Configuración de Prometheus y evidencia de alertas (JUP-045)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/apps/monitoring/prometheus/prometheus.yml.

[21] *Despliegue continuo: guía, integración en la PR 73 y ejecuciones de Actions*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/c3eb665aff7b5cfa53a5ad6a3795bd8c5e6c27e3/docs/deployment/dockerserver-cd.md.

[22] *Comparativa de GLM-5.2 y DeepSeek V4.1 Flash*. Documento en Google Drive. 19 de oct. de 2026. URL: https://drive.google.com/file/d/1e_X5dpDBifJrDUDKGyy9HeXpdNeBpSv3/view.

[23] *Informe de reevaluación tras el cambio de modelo*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/ded0b4346a4e11442448df980247c01f4367c867/docs/validation/model-reevaluation-report.md.

[24] *Banco de 28 preguntas: informes antes y después del arreglo de cortes*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/ded0b4346a4e11442448df980247c01f4367c867/docs/evidence.

[25] *Robustez ante datos incompletos antes del arreglo de cortes (informe)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/ded0b4346a4e11442448df980247c01f4367c867/docs/evidence/JUP-071-live-flash-r1-report.md.

[26] *Robustez ante datos incompletos tras el arreglo de cortes (informe)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/evidence/JUP-071-fix-r1-report.md.

[27] *Calidad del detector de anomalías (JUP-130)*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/ded0b4346a4e11442448df980247c01f4367c867/docs/evidence/JUP-130/JUP-130-anomalies.md.